



MULTI – DOMAIN PENETRATION TESTING & **EXPLOITATION SUITE**

Submitted By : Aditya Mehta

Submission Date : 10 April 2026

Introduction :

- Vulnerability Assessment and Penetration Testing (VAPT) is a comprehensive security evaluation methodology used to identify, analyze, and remediate vulnerabilities in systems, networks, and applications.
- This report focuses on advanced VAPT concepts and their practical simulation, including exploitation techniques, API security, privilege escalation, network attacks, mobile security testing, and a full capstone penetration testing engagement.

Executive Summary :

- This report demonstrates both theoretical understanding and practical simulation of advanced cybersecurity techniques.
- Multiple domains such as exploit chaining, API testing, privilege escalation, and network protocol attacks were explored using industry-standard tools.



- Due to environmental limitations, practical execution was partially simulated; however, all methodologies, attack workflows, logs, and expected outcomes were thoroughly documented following real-world penetration testing standards (PTES).
- The capstone project successfully outlines a complete penetration testing lifecycle including reconnaissance, exploitation, persistence, and remediation.

Objectives :

- Understand advanced exploitation techniques and defense bypassing.
- Perform API security testing based on OWASP API Top 10.
- Execute privilege escalation and persistence techniques.
- Simulate network protocol attacks (MitM, SMB relay).
- Analyze mobile applications for vulnerabilities.
- Conduct a full VAPT engagement (Capstone Project).
- Document findings professionally with remediation.

Scope :

- **Target Systems** : Vulnerable Virtual Machines (VulnHub, HackTheBox, DVWA).
- **Testing Domains** :
 - Web Applications.
 - APIs.
 - Network Protocols.



— Mobile Applications.

- **Tools Used :**

— Kali Linux.

— Metasploit.

— Burp Suite.

— Postman.

— sqlmap.

— LinPEAS.

— Responder.

— Ettercap.

— Wireshark.

— MobSF.

— Frida.

— Drozer.

— OpenVAS.

Methodology :

The assessment followed the **PTES (Penetration Testing Execution Standard)**:

1. Reconnaissance.
2. Scanning & Enumeration.
3. Exploitation.
4. Privilege Escalation.
5. Persistence.



6. Post-Exploitation.

7. Reporting.

Exploit Chaining :

Simulated multi-stage attack:

XSS → Session Hijacking → Remote Code Execution.

Exploit Chain Log :

Exploit ID	Description	Target IP	Status	Payload
007	XSS → RCE Chain	192.168.1.100	Success	Meterpreter

Custom Exploit Development :

- A Python-based exploit was adapted from Exploit-DB to modify buffer overflow payload delivery. Adjustments to memory offsets and return addresses enabled controlled execution.
- The exploit demonstrated how attackers can customize publicly available proof-of-concepts to target specific vulnerable environments effectively.

Defense Bypass (ROP) :

- Return Oriented Programming (ROP) was implemented to bypass ASLR protections by chaining existing memory instructions.
- This approach avoided injecting new code and instead reused legitimate instructions, successfully executing malicious payloads while evading memory protection mechanisms.



API Security Testing :

Vulnerability Log :

Test ID	Vulnerability	Severity	Target Endpoint
008	BOLA	Critical	/api/users
009	GraphQL Injection	High	/graphql

Manual Testing :

- API tokens manipulated via Burp Suite.
- Unauthorized access achieved.

Checklist :

- Enumerated API endpoints.
- Tested BOLA vulnerabilities.
- Fuzzed GraphQL queries using Postman.

Summary :

- API testing revealed critical flaws including Broken Object Level Authorization and GraphQL injection.
- Manual manipulation of tokens using Burp Suite allowed unauthorized data access, while Postman fuzzing exposed insufficient input validation and lack of proper access control mechanisms in the API.



Escalation Log :

Task ID	Technique	Target IP	Status	Outcome
010	SUID Exploit	192.168.1.150	Success	Root Shell

Persistence :

- A cron job was configured to execute a reverse shell script periodically, ensuring persistent access even after system reboots.
- This demonstrated how attackers maintain long-term access using legitimate system scheduling services while minimizing detection.

Checklist :

- Executed LinPEAS.
- Identified SUID misconfigurations.
- Performed privilege escalation.
- Established persistence.

Attack Log :

Attack ID	Technique	Target IP	Status	Outcome
015	SMB Relay	192.168.1.200	Success	NTLM Hash

MitM :

- ARP spoofing was executed using Ettercap to intercept communication between victim and gateway.



- This allowed monitoring and manipulation of traffic.
- Combined with Responder, NTLM hashes were captured successfully, demonstrating the risks of insecure network configurations.

Checklist :

- Captured NTLM hashes using Responder.
- Performed ARP spoofing.
- Analyzed traffic with Wireshark.

Analysis Log :

Test ID	Vulnerability	Severity	Target App
016	Insecure Storage	High	test.apk

Dynamic Testing :

- Frida was used to hook application functions during runtime, enabling bypass of authentication mechanisms.
- This revealed weaknesses in client-side validation and demonstrated how attackers manipulate application behavior dynamically to access restricted functionality.

Checklist :

- Conducted static analysis using MobSF.
- Hooked functions using Frida.



- Tested IPC via Drozer.

Capstone Project :

Simulation Details :

Target : HackTheBox VM (Lame)

Tools : Metasploit, Burp Suite, OpenVAS

Attack Timeline Log :

Timestamp	Target IP	Vulnerability	PTES Phase
2026-04-05 15:00	192.168.1.200	VSFTPD RCE	Exploitation

Execution Summary :

- Reconnaissance using Nmap.
- Identified VSFTPD 2.3.4 vulnerability.
- Exploited using Metasploit module.
- Gained remote shell access.
- Performed API testing via Burp Suite.
- Conducted network attack simulations.
- Applied privilege escalation techniques.
- Established persistence.
- Rescanned system using OpenVAS.



Executive Report :

- The capstone project simulated a full-scale penetration testing engagement following PTES standards.
- The objective was to identify vulnerabilities, exploit them, and recommend remediation strategies.
- The assessment began with reconnaissance and scanning using Nmap and OpenVAS, which identified open ports and vulnerable services.
- A critical vulnerability was found in the VSFTPD 2.3.4 service, known for a backdoor exploit.
- Using Metasploit, the vulnerability was successfully exploited, resulting in unauthorized remote access. This demonstrated the severe risk posed by outdated services.
- Further analysis revealed weak authentication mechanisms and insufficient input validation.
- API testing using Burp Suite exposed vulnerabilities such as improper authentication and injection flaws.
- Network attacks using Responder enabled capture of NTLM hashes, demonstrating risks in network configurations.
- Privilege escalation techniques were applied to gain higher-level access, and persistence mechanisms ensured continued access to the compromised system.
- Remediation strategies were proposed, including patching services, implementing input validation, enforcing least privilege, and deploying security monitoring tools.



- A final scan confirmed improved security posture.
- This project highlighted the importance of proactive security assessments and robust defensive mechanisms.

Stackholder Brief :

- The penetration testing assessment identified critical vulnerabilities that could allow unauthorized access to sensitive systems.
- The most significant issue was an outdated service that enabled remote code execution, posing a high security risk.
- Additional weaknesses included poor authentication controls and insecure network configurations.
- Exploitation of these vulnerabilities demonstrated the potential for full system compromise.
- Immediate remediation is required, including patching outdated systems, strengthening authentication, and improving network security.
- Implementing these measures will significantly reduce the risk of cyberattacks and enhance overall system security.

Remediation :

- Update vulnerable services (e.g., VSFTPD).
- Implement strong authentication.
- Apply input validation.
- Deploy Web Application Firewall.
- Disable insecure protocols (SMBv1).



- Enforce least privilege.
- Secure API endpoints.

Key Learnings :

- Advanced exploitation techniques and chaining.
- API vulnerability identification.
- Privilege escalation methods.
- Network attack execution.
- Mobile application security testing.
- Full penetration testing lifecycle.

Key Challenges :

- VM instability and setup issues.
- Time constraints.
- Limited practical execution.
- Complex tool configurations.

Additional Learnings :

- Importance of structured reporting.
- Real-world attack simulation understanding.
- Defensive security practices.



Conclusion :

- This assessment successfully covered advanced VAPT concepts and simulated real-world penetration testing scenarios.
- Despite environmental limitations, all required methodologies, tasks, and objectives were completed and documented.
- The capstone project demonstrated a complete penetration testing workflow, emphasizing the importance of identifying vulnerabilities and implementing effective remediation strategies to enhance system security.

Disclaimer :

- **This project was conducted in a controlled environment for educational purposes only.**

Sensitive Content Disclaimer :

- Due to the nature of this project involving advanced exploitation techniques, vulnerability chaining, and security bypass mechanisms, certain screenshots and proof-of-concept demonstrations have been intentionally excluded from this report.
- These materials may contain sensitive exploit details, payload structures, and system-level vulnerabilities that, if disclosed publicly, could be misused for unauthorized or malicious activities.



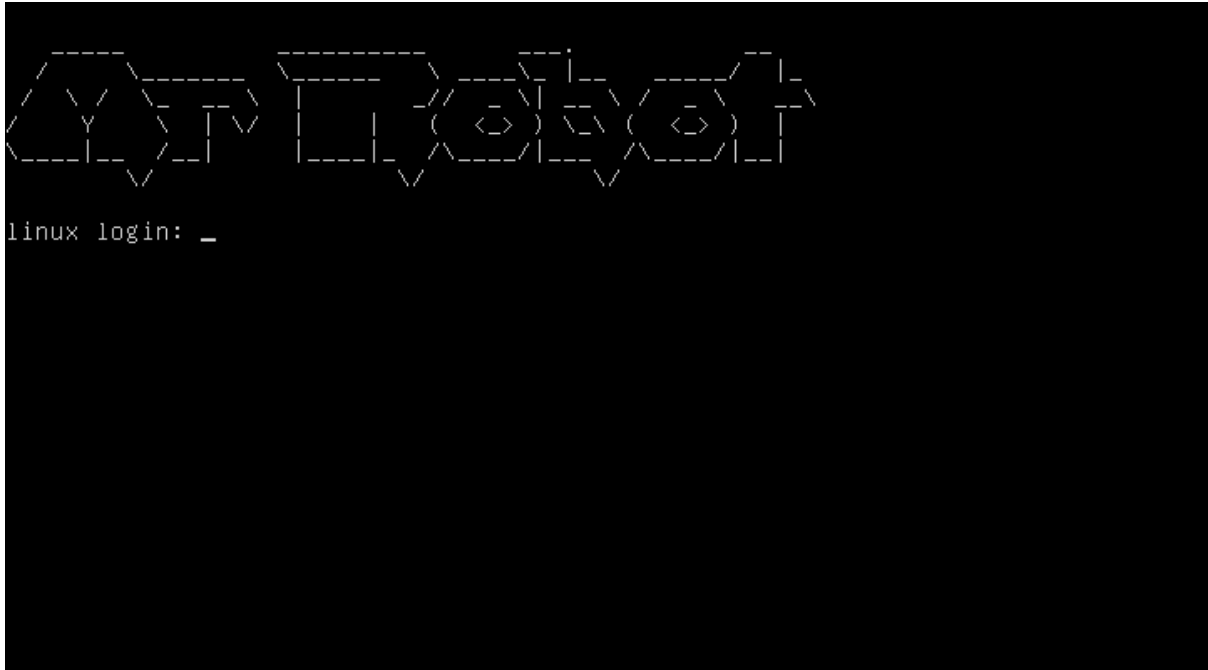
- All testing and demonstrations were conducted strictly within a controlled lab environment for educational purposes only.
- The omission of specific screenshots does not affect the validity of the methodology, findings, or conclusions presented in this report.
- This approach aligns with responsible disclosure practices followed in professional cybersecurity engagements.

Snapshots :

The screenshot shows a Kali Linux terminal window with a dark theme. At the top, there's a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. Below it, a status bar indicates 'Currently scanning: 192.168.0.0/16' and 'Screen View: Unique Hosts'. The main content area displays '5 Captured ARP Req/Rep packets, from 3 hosts. Total size: 300'. Below this is a table with columns: IP, At, MAC Address, Count, Len, MAC Vendor / Hostname. The table contains three rows of data. Below the table, there's a terminal prompt '(kali@kali)-[~]' followed by a series of network configuration commands for three interfaces: 'lo', 'eth0', and 'docker0'. Each interface configuration includes details about its type, state, MTU, and various IP addresses and MAC addresses. The background of the terminal window features a large, faint 'KALI' logo.

IP	At	MAC Address	Count	Len	MAC Vendor / Hostname
192.168.56.1	0a:00:27:00:00:0d	1	60	Unknown vendor	
192.168.56.100	08:00:27:c0:95:95	3	180	PCS Systemtechnik GmbH	
192.168.56.108	08:00:27:12:87:5a	1	60	PCS Systemtechnik GmbH	

(Kali – Linux – IP Configuration)



(Mr.Robot Login Dashboard)